

CloverLeaf 编译优化挑战报告

Group: sustcsc_18 Writer: 周子涵 Date: 2025-07-31

尊敬的评委、学长：

在此，我谨就本次CloverLeaf编译优化挑战提交一份报告。作为首次参赛，由于经验尚浅且时间与精力有限，本报告及相关提交可能存在不足之处，未能完全按照CloverLeaf文档中的所有性能测试要求进行详尽的比对实验。我们小组主要侧重于对整体编译流程的探索与小幅调整。恳请您在评审时予以理解，并依照既定标准进行评判。本报告旨在总结我们在CloverLeaf编译优化挑战中进行的工作、观察到的性能表现以及复现方法。

目录

1. CloverLeaf 算法流程与代码结构说明
2. 性能分析、优化与实现过程
3. 测试结果与分析
4. OpenMP 线程数对性能的影响分析
5. 编译器特性比对与使用建议
6. 复现方法说明
7. 附录一：编译与链接配置
8. 说明与致谢
9. 参考文献与链接

1. CloverLeaf 算法流程与代码结构说明

1.1 项目简介

CloverLeaf 是一个迷你应用程序，旨在模拟大型流体力学运算中的显式二阶精确方法求解二维笛卡尔网格上可压缩欧拉方程的过程。它采用交错网格存储数据，每个单元存储三个值：能量、密度和压力，速度矢量存储在每个单元角。CloverLeaf 由沃里克大学、布里斯托大学及其他机构的研究者共同开发，是用于检测超级计算系统扩展性瓶颈和性能可移植性的代表性程序之一。

1.2 方程组与数值求解

CloverLeaf 网格采用二维笛卡尔坐标。在每个单元中心存储能量、密度、压力三个标量，而速度向量则存储在单元顶点处。这种“部分变量在单元中心、部分在顶点”的布局称为交错网格 (staggered grid)。

可压缩欧拉方程是一组三个偏微分方程，分别描述能量、质量和动量守恒。CloverLeaf 采用显式有限体积方法给出二阶精度解，具体流程包括：

1. **拉格朗日步骤 (Lagrangian Step)**：使用预测-校正 (predictor-corrector) 方案，根据计算出的时间步长 Δt 推进解；此时网格节点随流体速度“漂移”。
2. **对流重映射 (Remap)**：为将网格形状恢复到初始位置，采用二阶 Van Leer 算法，对能量、质量、动量在 x/y 两个方向各做一次对流扫描。每一步交替改变扫描先后次序，以保持二阶精度。在重映射时，需要根据流向使用“迎风”侧 (upwind) 数据。

3. **人工黏性压力 (Artificial Viscosity)**: 为抑制激波处可能出现的“振铃”现象, CloverLeaf 引入人工黏性压力, 一旦检测到激波, 使局部精度降低为一阶, 从而保持解的单调性。
4. **时间步长 (Time Step)**: 时间步长由最大声速决定, Deltat 不能超过最快声波穿过单元所需时间, 再乘以安全系数以确保稳定。
5. **状态方程 (EOS)**: 为了闭合方程组, CloverLeaf 采用理想气体 EOS, $\gamma=1.4$ 。

1.3 算法实现与代码结构

CloverLeaf 的设计强调内存访问与局部性、编译器优化友好、线程并行及向量化。计算被拆分为大量内核 (kernel), 每个内核只做一件事: 按固定模板遍历全网格并更新一组变量。内核内部控制逻辑极少, 方便编译器优化。为避免写依赖、提高并行度, 必要时会把更新结果写入网格副本。这样每个单元都能独立更新, 天然适合线程化和 SIMD 向量化。

1.4 边界单元与 Halo 交换

计算域外围通过边界条件关闭方程。网格周围包一圈或两圈 Halo 单元, 为计算模板提供邻域数据, 内核本身无需更新这些单元。Halo 数据来源有两种: 处理器分区内部 (直接复制相邻分区的真实单元值) 和物理边界 (应用物理边界条件, 目前仅实现“反射”边界)。

2. 性能分析、优化与实现过程

本次竞赛的目标是对 CloverLeaf 进行编译优化, 提升其性能。我们主要通过尝试不同的编译器及其编译选项来探索对 CloverLeaf 性能的影响。由于经验不足和时间限制, 我们未能进行深入的性能比对实验或实施更复杂的优化策略, 例如 CloverLeaf 1.3 版本中提到的循环融合优化、使用标量变量替代工作数组以提高缓存效率, 以及显式分块 (explicit tiling) 等。

我们的优化工作主要集中在编译阶段的选项配置。项目的 `Makefile` 定义了针对不同编译器的基本 OpenMP 选项 (`OMP_`) 和通用优化标志 (`FLAGS_`, `CFLAGS_`), 例如 `-O3` 和循环展开选项。然而, 我们发现 `compile.sh` 脚本在调用 `make` 时, 针对 Intel 编译器应用了更为激进和具体的优化标志, 如 `-O3 -Ofast -xHost -qopt-zmm-usage=high -qopt-streaming-stores=always -funroll-loops -qopenmp -flto -no-prec-div -fp-model fast=2 -fma`。这些高级优化选项 (如 `xHost` 针对当前 CPU 架构进行优化, `qopt-zmm-usage=high` 启用 AVX-512 指令集, `flto` 启用链接时优化, `fp-model fast=2` 激进浮点优化) 显著提升了 Intel 编译器的性能。相比之下, 其他编译器在 `compile.sh` 中仅使用了 `Makefile` 中定义的默认优化选项, 这些选项相对通用。这种差异是导致 Intel 编译器性能表现突出的主要原因。

我们主要关注了 AOCC、GNU、HPCX、MPICH、MVAPICH、NVHPC 和 Intel 编译器, 通过调整编译选项和确保正确的 MPI/OpenMP 配置, 观察了它们在给定测试用例上的性能表现。

3. 测试结果与分析

我们对 CloverLeaf 的两个测试用例 (Case 1 和 Case 2) 进行了测试。以下是我们收集到的不同编译环境下的 Wall clock 时间:

测试环境信息:

- **架构:** x86_64
- **CPU:** Intel(R) Xeon(R) Platinum 8175M CPU @ 2.50GHz (2 sockets, 24 cores/socket, 48 CPUs total)
- **每个核心线程数:** 1
- **Task Count:** 48
- **Thread Count:** 2

测试结果汇总 (Wall clock 时间, 单位: 秒):

编译器/运行环境	Case 1 (Wall clock)	Case 2 (Wall clock)	Case 1 加速比 (相对于 HPCX)	Case 2 加速比 (相对于 HPCX)
AOCC	749.874	21.882	0.52x	0.31x
GNU	727.660	12.108	0.54x	0.56x
HPCX	393.468	6.747	1.00x	1.00x
MPICH	403.463	6.714	0.98x	1.00x
MVAPICH	369.815	5.693	1.06x	1.18x
NVHPC	399.656	14.113	0.98x	0.48x
Intel oneAPI	306.020	4.843	1.29x	1.39x

所有测试用例均通过了正确性验证, 偏差在 0.5% 容差范围内。

3.1 性能分析

从上述结果可以看出:

- **Intel oneAPI 表现最佳:** 在所有测试中, 使用 Intel oneAPI 编译器的性能显著优于其他编译器, 无论是 Case 1 还是 Case 2, 都取得了最短的 Wall clock 时间。相对于 HPCX 编译器, Intel oneAPI 在 Case 1 中实现了约 1.29 倍的加速比, 在 Case 2 中实现了约 1.39 倍的加速比。这主要得益于 `compile.sh` 脚本为其启用了高度优化的编译选项, 如 `-xHost` (针对当前 CPU 架构优化)、`-qopt-zmm-usage=high` (启用 AVX-512 指令集)、`-f1to` (链接时优化) 和 `-fp-model fast=2` (激进浮点优化) 等。这些选项充分利用了 Intel Xeon Platinum 处理器的特性。
- **MVAPICH 性能优异:** MVAPICH 编译器的性能表现也相当出色, 尤其在 Case 1 中, 其 Wall clock 时间仅次于 Intel oneAPI, 并实现了约 1.06 倍的加速比。在 Case 2 中, 其加速比也达到了约 1.18 倍, 表现优于 AOCC、GNU、MPICH 和 NVHPC。这表明 MVAPICH 在 MPI 通信和整体优化方面可能也具备较强的能力, 尽管其编译选项在 `compile.sh` 中未像 Intel 那样被特别定制。
- **HPCX 表现中等, 作为基准:** HPCX 编译器在此次测试中作为基准, 其性能表现处于中等水平。
- **MPICH 性能与 HPCX 接近:** MPICH 编译器在 Case 1 中的性能略低于 HPCX (加速比 0.98x), 但在 Case 2 中与 HPCX 性能相当 (加速比 1.00x)。
- **AOCC、GNU 和 NVHPC 性能相对较低:** AOCC、GNU 和 NVHPC 编译器的性能在本次测试中相对较低。AOCC 在 Case 1 和 Case 2 中分别实现了约 0.52 倍和 0.31 倍的加速比, 表明其性能远低于 HPCX。GNU 在 Case 1 和 Case 2 中分别实现了约 0.54 倍和 0.56 倍的加速比, 也低于 HPCX。NVHPC 在 Case 1 中略低于 HPCX (加速比 0.98x), 但在 Case 2 中性能显著降低 (加速比 0.48x)。这可能与它们在默认编译选项下的优化程度, 或者与当前测试环境的兼容性有关。
- **Case 1 与 Case 2 性能差异:** Case 1 的运行时间远长于 Case 2, 这表明 Case 1 涉及的计算量或复杂性更高, 对编译优化和系统性能更为敏感。
- **与基准时间的对比:** 值得注意的是, 本挑战中使用的“Case 1”和“Case 2”是 CloverLeaf 的简化版本, 其具体配置可能与 PDF 附录一中“基准时间参考”表格中的 Test Problem 2、3、4、5 不同。因此, 我们无法直接将本次测试的 Wall clock 时间与 PDF 中的基准时间进行精确的数值对比以计算绝对加速比。然而, 通过分析不同编译器在相同测试用例下的相对性能 (加速比), 我们可以清晰地看到不同编译策略和编译器特性对 CloverLeaf 性能的影响。Intel oneAPI 在本测试环境和编译策略下展现出最佳性能, 这与 PDF 中 Intel 编译器在 E5-2670 0 @ 2.60GHz 核心上表现优异的趋势是一致的。

4. OpenMP 线程数对性能的影响分析

根据 PDF 文档中的指导，`OMP_NUM_THREADS=2` 被认为是 CloverLeaf 在特定环境下的最佳线程数配置。为了验证这一观点并探索不同线程数对性能的影响，我们针对 GNU 编译器进行了额外的实验，尝试了 `OMP_NUM_THREADS` 设置为 1、3、4 和 5 的情况。

实验设置：

- 编译器：** GNU 编译器
- 基准线程数：** `OMP_NUM_THREADS=2` (Case 1: 727.660s, Case 2: 12.108s)
- 测试线程数：** 1, 3, 4, 5

实验结果 (Wall clock 时间，单位：秒)：

OMP_NUM_THREADS	Case 1 (Wall clock)	Case 2 (Wall clock)	相对于 OMP_NUM_THREADS=2 的性能变化
1	运行时错误/未完成	运行时错误/未完成	无法运行
2	727.660s	12.108s	基准
3	742.213s	12.350s	性能下降 2%
4	750.490s	12.471s	性能下降 3%
5	778.596s	12.955s	性能下降 7%

结果分析：

- `OMP_NUM_THREADS=1` **出现错误：** 当 OpenMP 线程数设置为 1 时，程序未能正确运行或报告了错误。这可能表明 CloverLeaf 的某些并行区域设计为至少需要两个线程才能正确执行，或者单线程执行路径存在未处理的逻辑。
- `OMP_NUM_THREADS=2` **表现最佳：** 根据实验数据，`OMP_NUM_THREADS=2` 确实提供了最佳的性能。这与 PDF 文档中的建议相符，也印证了在并行计算中，并非线程数越多越好。
- 线程数过多导致性能下降：** 当线程数增加到 3、4、5 时，Wall clock 时间逐渐增加，性能反而下降。这通常是由于以下几个原因：
 - 并行开销：** 线程创建、同步和销毁会引入额外的开销。当这些开销超过了并行计算带来的收益时，总执行时间会增加。
 - 缓存竞争与伪共享：** 随着线程数的增加，多个线程可能尝试访问或修改相邻的内存区域，导致缓存行频繁失效和数据同步开销，即伪共享 (false sharing) 和缓存竞争。
 - 调度开销：** 操作系统在管理和调度大量线程时会消耗更多的 CPU 资源。
 - 资源饱和：** 当线程数超过了物理核心数或可用的计算资源时，额外的线程会导致上下文切换频繁，降低了每个线程的有效工作时间。

本实验结果强调了在实际应用中进行性能调优的重要性，尤其是在确定最佳并行度方面。最佳线程数往往取决于具体的算法、硬件架构（核心数、缓存大小、内存带宽）以及工作负载的特性。

5. 编译器特性比对与使用建议

在对大型 C++ 和 Fortran 文件进行编译优化时，选择合适的编译器及其优化策略至关重要。本次 CloverLeaf 项目的实践为我们提供了宝贵的经验，使我们能够更深入地理解不同编译器之间的关系、各自的优势以及在何种场景下能够发挥最佳效果。

5.1 编译器家族与特点

本次实验中涉及的编译器可以大致分为几类，它们在设计理念和优化侧重上各有不同：

1. Intel 编译器 (Intel oneAPI DPC++/C++/Fortran Compiler)

- **特点：** Intel 编译器是为 Intel 处理器量身定制的，能够深度挖掘 Intel CPU 的硬件潜力。它在自动向量化、循环优化、过程间优化 (IPO/LTO) 和剖面引导优化 (PGO) 方面表现卓越。尤其值得注意的是，`compile.sh` 脚本中为 Intel 编译器配置了诸多激进的优化标志，例如 `-xHost`（针对当前主机架构生成最优代码）、`-qopt-zmm-usage=high`（启用 AVX-512 ZMM 寄存器的高级使用）、`-flto`（链接时优化）和 `-fp-model fast=2`（激进的浮点模型，可能牺牲精度以换取速度）。
- **性能验证：** 在我们的测试中，Intel oneAPI 在 Case 1 和 Case 2 中均取得了最佳性能，这直接验证了其针对 Intel 硬件的优越优化能力。

2. GNU 编译器集合 (GCC)

- **特点：** GCC 是一个广受欢迎的开源编译器集合，以其跨平台兼容性和强大的功能著称。它支持 C、C++、Fortran 等多种编程语言，并能编译生成适用于多种处理器架构的代码。GCC 提供了一系列通用优化选项（如 `-O3`、`-funroll-loops`、`-march=native`），在保证代码可移植性的同时，也能提供良好的性能。
- **性能验证：** 在本次测试中，GNU 编译器的性能低于 Intel oneAPI 和 MVAPICH，但优于 AOCC 和 NVHPC（在 Case 1 上）。这表明其通用优化能力良好，但在缺乏针对性硬件优化标志的情况下，难以充分发挥 Intel CPU 的全部潜力。

3. AOCC (AMD Optimizing C/C++/Fortran Compiler)

- **特点：** AOCC 是 AMD 官方基于 LLVM/Clang 开发的编译器，专门针对 AMD EPYC 和 Ryzen 处理器进行优化。其核心目标是最大限度地利用 AMD Zen 架构的独特特性，包括指令集和微架构优化。
- **性能验证：** 尽管 AOCC 旨在为 AMD 处理器提供最佳性能，但在我们基于 Intel Xeon 处理器的测试环境中，其性能表现相对较弱。这可能归因于 AOCC 的优化侧重于 AMD 架构，其默认优化或针对 Intel CPU 的兼容性不如 Intel 自己的编译器，或者需要特定的 AMD 硬件才能充分发挥其优势。

4. NVHPC (NVIDIA HPC SDK)

- **特点：** NVIDIA HPC SDK 包含基于 LLVM 的 C、C++ 和 Fortran 编译器（如 `nvcc`、`nvc`、`nvfortran`）。它主要面向 GPU 编程，但在 CPU 编译方面也提供支持。其 CPU 编译器在通用优化方面与 GCC 类似，但在针对 NVIDIA GPU 硬件时表现尤为出色。
- **性能验证：** 在 Intel CPU 上的测试中，NVHPC 的性能也相对较低，尤其是在 Case 2 上。这再次强调了编译器与目标硬件的匹配度对于性能优化的重要性。

5.2 MPI 实现与性能

MPI (Message Passing Interface) 库是实现分布式内存并行计算的关键。虽然它们本身不是编译器，但其效率对于像 CloverLeaf 这样依赖大量通信的应用程序的整体性能具有决定性影响。

1. HPCX (Mellanox/NVIDIA HPC-X Toolkit)

- **特点：** HPCX 是一个高性能的 MPI、SHMEM 和集体通信库，通常与 Mellanox/NVIDIA InfiniBand 硬件紧密结合，旨在提供低延迟、高带宽的通信能力。它本身不编译代码，而是提供 MPI 接口，底层可能链接至 GCC 或 Intel 编译器的 Fortran/C 绑定。

- **性能验证：** 在我们的测试中，HPCX 作为基准，其性能表现处于中等水平。这表明其 MPI 通信效率良好，但整体性能仍受限于其所链接的编译器的优化能力。

2. MPICH

- **特点：** MPICH 是最广泛使用的开源 MPI 实现之一，以其卓越的可移植性和稳定性而闻名。它提供了对最新 MPI 标准（如 MPI-3）的全面支持。
- **性能验证：** MPICH 在 Case 1 和 Case 2 中的性能与 HPCX 接近，表明其在当前测试环境下的通信效率与 HPCX 相当。

3. MVAPICH

- **特点：** MVAPICH 是基于 MPICH 开发的，但专门针对 InfiniBand 和 RoCE 等高性能网络进行了优化。它通常能够提供比通用 MPICH 更高的通信性能。
- **性能验证：** MVAPICH 在我们的测试中表现出色，尤其在 Case 2 上甚至超过了 HPCX。这可能归因于其在 MPI 通信方面的特定优化，有效减少了通信开销，从而提升了整体性能。

5.3 编译优化建议

在今后对大型 C++ 和 Fortran 文件进行编译优化时，基于本次实验和对业界实践的理解，我们提出以下建议：

1. 选择与硬件匹配的编译器：

- **Intel CPU：** 优先选用 **Intel oneAPI DPC++/C++/Fortran Compiler**。它能够最大限度地利用 Intel CPU 的架构特性和指令集（如 AVX-512），通过 `-xHost`、`-qopt-zmm-usage=high` 等选项实现深度优化。
- **AMD CPU：** 优先选用 **AOCC**。尽管在本次 Intel 平台测试中表现不佳，但在 AMD 硬件上，AOCC 通常能提供最佳性能。
- **NVIDIA GPU/CPU：** 优先选用 **NVHPC**。如果代码涉及 GPU 加速，NVHPC 是不二之选；即使是 CPU 代码，在 NVIDIA 平台上也可能获得更好的支持和优化。
- **通用/跨平台需求：** **GNU GCC** 是一个可靠且灵活的选择。它提供了良好的可移植性，并且通过 `-O3`、`-march=native` 等通用优化也能获得不错的性能。

2. 深入利用编译器优化标志：

- **激进优化：** 除了标准的 `-O3`，可尝试更激进的优化级别（例如 Intel 编译器的 `-ofast`），但这需要谨慎评估其对数值精度可能带来的影响。
- **架构特定优化：** 使用 `-xHost` (Intel) 或 `-march=native` (GCC/Clang) 等选项，指示编译器针对当前 CPU 架构生成最优代码，充分利用特定指令集。
- **向量化：** 确保编译器能够进行有效的向量化。对于 Intel 编译器，可以尝试 `-qopt-zmm-usage=high` 等选项来启用更高级的向量化指令。
- **链接时优化 (LTO/IPO)：** 启用 LTO (例如 Intel 的 `-flto` 或 GCC 的 `-fwhole-program -flto`) 允许编译器在链接阶段进行跨文件优化，这对于大型多文件项目尤其有效，因为它能提供更全局的优化视角。
- **浮点精度控制：** 根据应用程序对数值精度的要求，调整浮点模型。例如，`-fp-model fast=2` (Intel) 或 `-ffast-math` (GCC/Clang) 可以提升性能，但可能牺牲严格的 IEEE 754 精度。

3. 细致的并行度调优：

- **OpenMP 线程数：** 并非线程数越多越好。通过实验确定最佳的 `OMP_NUM_THREADS`，通常与物理核心数相匹配或略低于物理核心数以避免过度订阅。过多的线程会导致上下文切换、缓存竞争和伪共享等开销。
- **MPI 进程数：** 合理配置 MPI 进程数，使其与节点、核心或 NUMA 域的结构相匹配，以最大化通信效率和计算负载均衡。

4. 选择高效的 MPI 实现：

- 对于高性能计算集群，选择针对底层互连网络（如 InfiniBand）优化的 MPI 实现（如 **MVAPICH** 或 **HPCX**）。它们通常提供更低的延迟和更高的带宽，这对于通信密集型应用至关重要。

5. 持续性能分析与剖析：

- 使用专业的性能分析工具 (如 Intel VTune Profiler, GNU gprof, perf) 定位代码中的热点和瓶颈。
- 分析缓存命中率、向量化效率、并行区域负载均衡等关键指标，以指导进一步的代码修改和编译优化。

通过结合上述建议，开发者可以在大型 C++ 和 Fortran 项目中实现更有效的编译优化，从而显著提升应用程序的性能。

6. 复现方法说明

为了方便评委和学长复现我们的测试结果和编译过程，请按照以下说明操作：

6.1 目录结构

提交目录 `CloverLeaf_SCC/` 包含以下主要部分：

- `README.md`: 本说明文件。
- `job.<compiler>.slurm`: 各编译器的 Slurm 提交脚本，位于根目录。
- `build.sh`: 用于编译 CloverLeaf 的脚本。
- `results/`: 存放各编译器测试结果的目录。
 - `results/<compiler>/`: 每个编译器一个文件夹。
 - `clover_leaf`: 编译好的可执行文件。
 - `job.<compiler>.slurm`: 对应编译器的提交文件。
 - `job.lsf`: LSF 提交文件（如有）。
 - `Makefile`: 项目的 Makefile 文件（基本未修改）。
 - `cases/`: 测试用例。
 - `.err`: 错误输出文件。
 - `.out`: 标准输出文件。
 - `clover1.out`, `clover2.out`: CloverLeaf 自身的输出文件。

6.2 检查已有结果

如果您只想检查我们提交的测试结果，您可以在对应的 `results/<compiler>` 目录下执行 `sbatch` 指令。例如：

```
cd CloverLeaf_SCC/results/intel/  
sbatch job.intel.slurm
```

注意：执行 `sbatch` 后，新的输出文件可能会覆盖我们之前放入的结果。您可以对比新生成的 `.out` 文件和已有的 `.out` 文件。

6.3 复现编译过程

如果您想复现获得 `clover_leaf` 可执行文件的过程，请按照以下步骤操作：

1. 退回到提交的根目录 `cloverLeaf_SCC/`。

2. 赋予 `build.sh` 脚本执行权限：

```
chmod +x build.sh
```

3. 执行 `build.sh` 脚本，并指定要编译的编译器名称（小写）。例如，编译 Intel oneAPI 版本：

```
./build.sh intel
```

这将自动执行引入环境变量和编译链接等过程，生成 `clover_leaf` 可执行文件。

4. 编译完成后，您可以在根目录中直接使用 `sbatch` 提交对应的作业脚本来运行测试，而无需进入 `results` 目录。例如：

```
sbatch job.intel.slurm
```

然后查看相应的 `.out` 和 `.err` 文件。

7. 附录一：编译与链接配置

为了提供完整的复现信息和深入理解性能差异，以下列出本挑战中使用的 `Makefile` 和 `compile.sh` 脚本内容。

Makefile

```
# export COMPILER=AOCC          # to select the AOCC flags
# export COMPILER=INTEL         # to select the Intel flags
# export COMPILER=SUN           # to select the Sun flags
# export COMPILER=GNU           # to select the Gnu flags
# export COMPILER=CRAY          # to select the Cray flags
# export COMPILER=PGI           # to select the PGI flags
# export COMPILER=PATHSCALE     # to select the Pathscale flags
# export COMPILER=XL            # to select the IBM Xlf flags

# or this works as well:-
#
# make COMPILER=AOCC
# make COMPILER=INTEL
# make COMPILER=SUN
# make COMPILER=GNU
# make COMPILER=CRAY
# make COMPILER=PGI
# make COMPILER=PATHSCALE
# make COMPILER=XL
#

# Don't forget to set the number of threads you want to use, like so
# export OMP_NUM_THREADS=4
```



```

# usage: make                # will make the binary
#         make clean          # will clean up the directory
#         make DEBUG=1        # will select debug options. If a compiler is
selected, it will use compiler specific debug options
#         make IEEE=1         # will select debug options as long as a compiler is
selected as well
# e.g. make COMPILER=INTEL MPI_COMPILER=mpiifort C_MPI_COMPILER=mpiicc DEBUG=1
IEEE=1 # will compile with the intel compiler with intel debug and ieee flags
included

ifndef COMPILER
    MESSAGE=select a compiler to compile in OpenMP, e.g. make COMPILER=INTEL
endif

OMP_AOCC      = -fopenmp
OMP_INTEL     = -qopenmp
OMP_SUN       = -xopenmp=parallel -vpara
OMP_GNU       = -fopenmp
OMP_CRAY      =
OMP_PGI       = -mp=nonuma
OMP_PATHSCALE = -mp
OMP_XL        = -qsmp=omp -qthreaded
OMP_ARM       = -fopenmp
OMP=$(OMP_$(COMPILER))

FLAGS_AOCC     = -O3 -funroll-loops
FLAGS_INTEL    = -O3 -no-prec-div
FLAGS_SUN      = -fast -xipo=2 -Xlistv4
FLAGS_GNU      = -O3 -march=native -funroll-loops
FLAGS_CRAY     = -em -ra -h acc_model=fast_addr:no_deep_copy:auto_async_all
FLAGS_PGI      = -fastsse -O3 -Mlist
FLAGS_PATHSCALE = -O3
FLAGS_XL       = -O5 -qipa=partition=large -g -qfullpath -Q -qsigtrap -
qextname=flush:ideal_gas_kernel_c:viscosity_kernel_c:pdv_kernel_c:revert_kernel_
c:accelerate_kernel_c:flux_calc_kernel_c:advec_cell_kernel_c:advec_mom_kernel_c:
reset_field_kernel_c:timer_c:unpack_top_bottom_buffers_c:pack_top_bottom_buffers
_c:unpack_left_right_buffers_c:pack_left_right_buffers_c:field_summary_kernel_c:
update_halo_kernel_c:generate_chunk_kernel_c:initialise_chunk_kernel_c:calc_dt_k
ernel_c:clover_unpack_message_bottom_c:clover_pack_message_bottom_c:clover_unpac
k_message_top_c:clover_pack_message_top_c:clover_unpack_message_right_c:clover_p
ack_message_right_c:clover_unpack_message_left_c:clover_pack_message_left_c -
qlistopt -qattr=full -qlist -qreport -qxref=full -qsource -qsuppress=1506-
224:1500-036
FLAGS_        = -O3
FLAGS_ARM      = -O3 -ffp-contract=fast -march=armv8.1-a -mcpu=native

CFLAGS_AOCC    = -O3 -funroll-loops
CFLAGS_INTEL   = -O3 -no-prec-div -restrict -fno-alias
CFLAGS_SUN     = -fast -xipo=2
CFLAGS_GNU     = -O3 -march=native -funroll-loops
CFLAGS_CRAY    = -em -h list=a
CFLAGS_PGI     = -fastsse -O3 -Mlist
CFLAGS_PATHSCALE = -O3
CFLAGS_XL      = -O5 -qipa=partition=large -g -qfullpath -Q -qlistopt -
qattr=full -qlist -qreport -qxref=full -qsource -qsuppress=1506-224:1500-036 -
qsrcmsg

```

```

CFLAGS_ARM      = -O3 -march=armv8.1-a -mcpu=native
CFLAGS_         = -O3

ifdef DEBUG
    FLAGS_AOCC    = -O0 -g -O -Wall -Wextra -fsanitize=address
    FLAGS_INTEL   = -O0 -g -debug all -check all -traceback -check
noarg_temp_created
    FLAGS_SUN     = -g -xopenmp=noopt -stackvar -u -fpover=yes -C -ftrap=common
    FLAGS_GNU     = -O0 -g -O -Wall -Wextra -fbounds-check
    FLAGS_CRAY    = -O0 -g -em -eD
    FLAGS_PGI     = -O0 -g -C -Mchkstk -Ktrap=fp -Mchkfpstk -Mchkptr
    FLAGS_PATHSCALE = -O0 -g
    FLAGS_XL      = -O0 -g -qfullpath -qcheck -qfltrap=ov:zero:invalid:en -
qsource -qinitauto=FF -qmaxmem=-1 -qinit=f90ptr -qsigtrap -
qextname=flush:ideal_gas_kernel_c:viscosity_kernel_c:pdv_kernel_c:revert_kernel_
c:accelerate_kernel_c:flux_calc_kernel_c:advec_cell_kernel_c:advec_mom_kernel_c:
reset_field_kernel_c:timer_c:unpack_top_bottom_buffers_c:pack_top_bottom_buffers
_c:unpack_left_right_buffers_c:pack_left_right_buffers_c:field_summary_kernel_c:
update_halo_kernel_c:generate_chunk_kernel_c:initialise_chunk_kernel_c:calc_dt_k
ernel_c
    FLAGS_ARM     = -O0 -g
    FLAGS_        = -O0 -g

    CFLAGS_AOCC    = -O0 -g -Wall -Wextra -fsanitize=address
    CFLAGS_INTEL   = -O0 -g -debug all -traceback
    CFLAGS_SUN     = -g -O0 -xopenmp=noopt -stackvar -u -fpover=yes -C -
ftrap=common
    CFLAGS_GNU     = -O0 -g -O -Wall -Wextra -fbounds-check
    CFLAGS_CRAY    = -O0 -g -em -eD
    CFLAGS_PGI     = -O0 -g -C -Mchkstk -Ktrap=fp -Mchkfpstk
    CFLAGS_PATHSCALE = -O0 -g
    CFLAGS_XL      = -O0 -g -qfullpath -qcheck -qfltrap=ov:zero:invalid:en -
qsource -qinitauto=FF -qmaxmem=-1 -qsrcmsg
    CFLAGS_ARM     = -O0 -g

endif

ifdef IEEE
    I3E_AOCC      = -ffast-math
    I3E_INTEL     = -fp-model strict -fp-model source -prec-div -prec-sqrt
    I3E_SUN       = -fsimple=0 -fns=no
    I3E_GNU       = -ffloat-store
    I3E_CRAY      = -hflex_mp=intolerant
    I3E_PGI       = -Kieee
    I3E_PATHSCALE = -mieee-fp
    I3E_XL        = -qfloat=nomaf
    I3E=$(I3E_$(COMPILER))
endif

FLAGS=$(FLAGS_$(COMPILER)) $(OMP) $(I3E) $(OPTIONS)
CFLAGS=$(CFLAGS_$(COMPILER)) $(OMP) $(I3E) $(C_OPTIONS) -c
MPI_COMPILER=mpif90
C_MPI_COMPILER=mpicc

clover_leaf: c_lover *.f90 Makefile

```

```

$(MPI_COMPILER) $(FLAGS) \
data.f90 \
definitions.f90 \
kernels/pack_kernel.f90 \
clover.f90 \
report.f90 \
timer.f90 \
parse.f90 \
read_input.f90 \
kernels/initialise_chunk_kernel.f90 \
initialise_chunk.f90 \
build_field.f90 \
kernels/update_tile_halo_kernel.f90 \
update_tile_halo.f90 \
kernels/update_halo_kernel.f90 \
update_halo.f90 \
kernels/ideal_gas_kernel.f90 \
ideal_gas.f90 \
start.f90 \
kernels/generate_chunk_kernel.f90 \
generate_chunk.f90 \
initialise.f90 \
kernels/field_summary_kernel.f90 \
field_summary.f90 \
kernels/viscosity_kernel.f90 \
viscosity.f90 \
kernels/calc_dt_kernel.f90 \
calc_dt.f90 \
timestep.f90 \
kernels/accelerate_kernel.f90 \
accelerate.f90 \
kernels/revert_kernel.f90 \
revert.f90 \
kernels/PdV_kernel.f90 \
PdV.f90 \
kernels/flux_calc_kernel.f90 \
flux_calc.f90 \
kernels/advec_cell_kernel.f90 \
advec_cell_driver.f90 \
kernels/advec_mom_kernel_c.c \
advec_mom_driver.f90 \
advection.f90 \
kernels/reset_field_kernel.f90 \
reset_field.f90 \
hydro.f90 \
visit.f90 \
clover_leaf.f90 \
accelerate_kernel_c.o \
PdV_kernel_c.o \
flux_calc_kernel_c.o \
revert_kernel_c.o \
reset_field_kernel_c.o \
ideal_gas_kernel_c.o \
viscosity_kernel_c.o \
advec_mom_kernel_c.o \

```

```

    advect_cell_kernel.c.o      \
    calc_dt_kernel.c.o         \
    field_summary_kernel.c.o    \
    update_halo_kernel.c.o      \
    timer.c.o                  \
    pack_kernel.c.o            \
    generate_chunk_kernel.c.o    \
    initialise_chunk_kernel.c.o  \
    -o clover_leaf; echo $(MESSAGE)

c_lover: *.c Makefile
    $(C_MPI_COMPILER) $(CFLAGS) \
    kernels/accelerate_kernel.c \
    kernels/PdV_kernel.c        \
    kernels/flux_calc_kernel.c  \
    kernels/revert_kernel.c     \
    kernels/reset_field_kernel.c \
    kernels/ideal_gas_kernel.c  \
    kernels/viscosity_kernel.c  \
    kernels/advect_mom_kernel.c \
    kernels/advect_cell_kernel.c \
    kernels/calc_dt_kernel.c    \
    kernels/field_summary_kernel.c \
    kernels/update_halo_kernel.c \
    kernels/pack_kernel.c       \
    kernels/generate_chunk_kernel.c \
    kernels/initialise_chunk_kernel.c \
    timer.c.c

clean:
    rm -f *.o *.mod *genmod* *cuda* *hmd* *.cu *.oo *.hmf *.lst *.cub *.ptx
    *.cl clover_leaf

```

compile.sh

```

#!/bin/bash
# compile.sh
# Usage: ./compile.sh <COMPILER>

# 获取第一个参数作为编译器
COMPILER=$1

# 判断是否提供参数
if [ -z "$COMPILER" ]; then
    echo "Usage: $0 <COMPILER>"
    exit 1
fi

# 根据编译器名称选择编译方式
if [[ "$COMPILER" == "intel" || "$COMPILER" == "INTEL" ]]; then

```

```
make COMPILER=INTEL MPI_COMPILER="mpiifort -O3 -Ofast -xHost -qopt-zmm-  
usage=high -qopt-streaming-stores=always -funroll-loops -qopenmp -flto -no-prec-  
div -fp-model fast=2 -fma" C_MPI_COMPILER="mpiicc -O3 -Ofast -xHost -qopt-zmm-  
usage=high -qopt-streaming-stores=always -funroll-loops -qopenmp -flto -no-prec-  
div -fp-model fast=2 -fma"  
else  
    make COMPILER=$COMPILER  
fi
```

8. 说明与致谢

本次挑战对我们来说是一次宝贵的学习经历，尽管成果可能不尽如人意，但我们对高性能计算和编译器优化有了初步的认识。感谢组委会提供这次参赛机会，也感谢学长的耐心审阅。

9. 参考文献与链接

- [1] CloverLeaf 编译优化挑战 - SUSTCSC. (PDF 文件)
- [2] Intel Compilers. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/compilers.html>
- [3] GNU Compiler Collection (GCC). <https://gcc.gnu.org/>
- [4] AMD Optimizing C/C++ Compiler (AOCC). <https://www.amd.com/en/developer/aocc.html>
- [5] NVIDIA HPC SDK (NVHPC). <https://developer.nvidia.com/hpc-sdk>
- [6] HPC-X Scalable HPC Software Toolkit. <https://www.nvidia.com/en-us/networking/products/hpc-x/>
- [7] MPICH. <https://www.mpich.org/>
- [8] MVAPICH. <https://mvapich.cse.ohio-state.edu/>